

Allowing `static_assert(false)`

Document #: P2593R1
Date: 2022-11-08
Project: Programming Language C++
Audience: CWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 The Actual Rule](#)
 - [2.2 Workarounds](#)
 - [2.3 Valid Workaround](#)
 - [2.4 History](#)
- [3 Proposal](#)
 - [3.1 Wording](#)
 - [3.2 Implementation Experience](#)
 - [3.3 Alternative Designs](#)
- [4 Acknowledgements](#)
- [5 References](#)

§ 1 Revision History

Since [[P2593R0](#)], added some alternative approaches.

§ 2 Introduction

Consider the two functions below:

Runtime	Compile time
<pre> template <class T> void do_something(T t) { if (is_widget(t)) { use_widget(t); } else if (is_gadget(t)) { use_gadget(t); } else { assert(false); } } </pre>	<pre> template <class T> void do_something(T t) { if constexpr (is_widget<T>) { use_widget(t); } else if constexpr (is_gadget<T>) { use_gadget(t); } else { static_assert(false); } } </pre>

The code on the left is fairly unremarkable. Having a bunch of runtime conditions that are intended to be exhaustive with a trailing `assert(false);` is not an uncommon pattern, in C++ or in C. It's just: if I get here, it's a bug, so assert here.

The code on the right is also fairly unremarkable, simply the compile-time version of the code on the left, where all the checks can be done statically. This is safer, since we can verify *at compile time* that we actually covered all the cases, ensuring that any bugs are compile errors rather than runtime errors. This is great. People expect the code on the right to do roughly the same as the code on the left, just reporting the error earlier.

Except while the code on the left works, the code on the right is ill-formed, no diagnostic required. And, in practice, all compilers diagnose this case immediately.

But users still want to write code using this structure. Sure, in this particular case, they can rewrite it:

Desired structure (IFNDR)	Working structure (✓)
<pre> template <class T> void do_something(T t) { if constexpr (is_widget<T>) { use_widget(t); } else if constexpr (is_gadget<T>) { use_gadget(t); } else { static_assert(false); } } </pre>	<pre> template <class T> void do_something(T t) { if constexpr (is_widget<T>) { use_widget(t); } else { static_assert(is_gadget<T>); use_gadget(t); } } </pre>

But in other cases that might not be possible. Such as wanting to ensure that the primary template is not instantiated - there's no convenient, ready-made condition to put here:

```

template <class T>
struct should_be_specialized {
    static_assert(false, "this isn't the specialization you're looking for");
};

```

§ 2.1 The Actual Rule

The actual rule that `static_assert(false)` runs afoul of is 13.8 [temp.res]/6:

- 6 The validity of a template may be checked prior to any instantiation.

[Note 3: Knowing which names are type names allows the syntax of every template to be checked in this way. — *end note*]

The program is ill-formed, no diagnostic required, if:

- (6.1) — no valid specialization can be generated for a template or a substatement of a constexpr if statement within a template and the template is not instantiated, or
- (6.2) — ...

Since there is, indeed, no valid specialization that would case `static_assert(false)` to be valid, the program is ill-formed, no diagnostic required.

But... that's the goal of the code, to not have a valid specialization in this case! It's just that the obvious solution to the goal happens to cause us to not even have a program.

§ 2.2 Workarounds

As a result of `static_assert(false)` not working, people turn to workarounds. Which are, basically: how can I write `false` in a sufficiently complex way so as to confuse the compiler?

Some of these workarounds actually still do run afoul of the above rule, but compilers aren't smart enough to diagnose them, so they Just Happen To Work. Checks like:

- `static_assert(sizeof(T) == 0);`
- `static_assert(sizeof(T*) == 0);`
- `[]<bool flag=false>(){ static_assert(flag); }();` from [this answer](#).

These are all terrible. And also wrong. But they happen to get the job done so people use them.

§ 2.3 Valid Workaround

A more valid workaround is to have some template that is always false (e.g. from [this answer](#)):

```
static_assert(always_false<T>::value);
```

This is also terrible, but as long as the condition is dependent, there could hypothetically be some instantiation that is `true`, and thus we don't run afoul of the [temp.res] rule. So it is technically valid. This is really the only real workaround for this problem, and is something that people have to be taught explicitly as a workaround.

§ 2.4 History

There was a proposal to add this to the standard library [P1830R1]. The problem with having a library solution to this problem is that you have to handle various kinds. It's not enough to have `always_false_v<Type>` since that would require having to write `always_false_v<integral_constant<decltype(v), v>>` if what you happen to have is a value. And if all you have is a class template, this is even worse. You need [P1985R1] simply to even claim to be able to solve this problem in the library.

This was followed up by a language proposal to make `static_assert` more dependent [P1936R0] - which proposed `static_assert<T>(false)`. This has the same issues with kind as the library solution, but also is just adding more stuff to `static_assert` that still needs to be taught. Sometimes you need this extra stuff, sometimes you don't?

§ 3 Proposal

The proposal here is quite simple. `static_assert(false)` should just work.

§ 3.1 Wording

Change 9.1 [dcl.pre]/10:

- ¹⁰ In a *static_assert-declaration*, the *constant-expression* is contextually converted to `bool` and the converted expression shall be a constant expression ([expr.const]). If the value of the expression when so converted is `true` or the expression is evaluated in a non-instantiated template, the declaration has no effect. Otherwise, the program is ill-formed, and the resulting diagnostic message ([intro.compliance]) should include the text of the *string-literal*, if one is supplied.

[Example 3:

```
static_assert(sizeof(int) == sizeof(void*), "wrong pointer size");
static_assert(sizeof(int[2]));           // OK, narrowing allowed

+ template <class T>
+ void f(T t) {
+   if constexpr (sizeof(T) == 4) {
+     use(t);
+   } else {
+     static_assert(false, "must be size 4");
+   }
+ }
+
+ void g(char c) {
+   f(c); // error: must be size 4
+ }
```

— end example]

Change 13.8 [temp.res]/6:

- 6 The validity of a template may be checked prior to any instantiation.

[Note 3: Knowing which names are type names allows the syntax of every template to be checked in this way. — *end note*]

The program is ill-formed, no diagnostic required, if:

- (6.1) — ignoring all *static-assert-declarations*, no valid specialization can be generated for a template or a substatement of a constexpr if statement within a template and the template is not instantiated, or
- (6.2) — ...

This sidesteps the question of whether it's *just* `static_assert(false)` that should be okay or `static_assert(0)` or `static_assert(1 - 1)` or `static_assert(not_quite_dependent_false)`. anything else. Just, all `static_assert` declarations should be delayed until the template (or appropriate specialization or constexpr if substatement thereof) is actually instantiated.

If the condition is false, we're going to get a compiler error anyway. And that's fine! But let's just actually fail the program when it's actually broken, and not early.

§ 3.2 Implementation Experience

I implemented this in both EDG and Clang. In both compilers, it's basically a two line code change: simply don't try to diagnose `static_assert` declarations if we're still in a template dependent context. Wait until they're instantiated.

§ 3.3 Alternative Designs

While this proposal simply treats the condition of `static_assert` as always dependent, there were a few other potential approaches to this problem.

The first is rather than treating *all* conditions as dependent, simply allow `static_assert(false)` directly. Or maybe not just `false`, literally the token `false`, but maybe also the token `0`. Or some other specific subset of expressions that is extremely `false`. Perhaps then `static_assert(false)` and `static_assert(0)` would delay triggering until instantiation, but something like `static_assert(condition)` where `condition` is an `inline constexpr bool` variable would trigger immediately. Really just allowing `static_assert(false)` is the primary goal, so this would be good enough, but it's pretty weird to special-case certain expressions like this. And I'm not even sure what the right set of "falsey-enough" expressions would be, or how to specify it. So that doesn't seem like a better alternative.

Another approach would be to treat `static_assert("error")` differently. This is technically a valid declaration today, although it is basically meaningless, equivalent to writing `static_assert(true)`. We could treat `static_assert(string-literal)` as the kind of unconditional diagnostic on instantiation that I'm going for with `static_assert(false)`. This certainly seems interesting at first glance, but it has two problems with it:

1. It's something people would have to be taught to do, whereas `static_assert(false)` is already what people initially try
2. At some point, we will extend `static_assert` to allow richer messages. Not just a string-literal, maybe a full `std::string` that is a result of a call to `std::format`? If we ever do that, then `static_assert(std::format("error"))` wouldn't really be able to have this special behavior.

The nice thing about simply allowing all conditions to be treated as dependent is that it's straightforward to understand, specify, and implement, doesn't cut off future evolution, and is already what people expect to work. I'm not sure that there's another design that satisfies all of these criteria.

§ 4 Acknowledgements

Thanks to Daveed Vandevoorde for helping with the implementation.

Thanks to all the people over the years to whom I've had to explain why `static_assert(false);` doesn't work and what they have to write instead for not immediately destroying their computers and switching to Rust.

§ 5 References

[P1830R1] Ruslan Arutyunyan. 2019-10-07. `std::dependent_false`.

<https://wg21.link/p1830r1>

[P1936R0] Ruslan Arutyunyan. 2019-10-07. Dependent Static Assertion.

<https://wg21.link/p1936r0>

[P1985R1] Gašper Ažman, Mateusz Pusz, Colin MacLean, Bengt Gustafsonn. 2020-05-15. Universal template parameters.

<https://wg21.link/p1985r1>

[P2593R0] Barry Revzin. 2022-05-21. Allowing `static_assert(false)`.

<https://wg21.link/p2593r0>